

TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN

-----***-----



Chủ đề: QUẢN LÝ ĐÀO TẠO ĐẠI HỌC

Giảng viên: TS. PGS. Nguyễn Duy hàm

Dương Hạo Quang

3124411244

Thành phố Hồ Chí Minh, ngày 22 tháng 9 năm 2026

I. PHẦN A – PHÂN TÍCH VÀ THIẾT KẾ CƠ SỞ DỮ LIỆU

CÂU 1. XÁC ĐỊNH THỰC THỂ, THUỘC TÍNH, KHÓA CHÍNH VÀ KHÓA ỨNG VIÊN

- Hệ thống gồm 8 thực thể chính (entity) mô tả các đối tượng độc lập trong bài toán, và 3 thực thể liên kết (associative entity) sinh ra từ các mối quan hệ nhiều–nhiều có mang thuộc tính riêng. Khóa chính của mỗi thực thể liên kết là khóa tổ hợp (composite key) gồm các khóa ngoại tham chiếu tới hai thực thể liên quan.

A. Các thực thể chính

1. KHOA

Thuộc tính	Vai trò / Ghi chú
MaKhoa	Khóa chính (PK)
TenKhoa	Khóa ứng viên (duy nhất, không dùng làm PK)

2. BỘ MÔN

Thuộc tính	Vai trò / Ghi chú
MaBM	Khóa chính (PK)
TenBM	Khóa ứng viên
MaKhoa	Khóa ngoại (FK → KHOA)
MaTruongBM	Khóa ngoại (FK → GIẢNG VIÊN) — thể hiện quan hệ "Là trưởng bộ môn"

3. GIẢNG VIÊN

Thuộc tính	Vai trò / Ghi chú
MaGV	Khóa chính (PK)
TenGV	Thuộc tính mô tả
MaBM	Khóa ngoại (FK → BỘ MÔN)

4. NGÀNH

Thuộc tính	Vai trò / Ghi chú
MaNganh	Khóa chính (PK)
TenNganh	Khóa ứng viên

Thuộc tính	Vai trò / Ghi chú
MaKhoa	Khóa ngoại (FK → KHOA) — quan hệ giả định thêm, đề không nêu rõ

5. LỚP HÀNH CHÍNH

Thuộc tính	Vai trò / Ghi chú
MaLop	Khóa chính (PK)
TenLop	Thuộc tính mô tả
MaNganh	Khóa ngoại (FK → NGÀNH)

6. SINH VIÊN

Thuộc tính	Vai trò / Ghi chú
MaSV	Khóa chính (PK)
TenSV	Thuộc tính mô tả
MaLop	Khóa ngoại (FK → LỚP HÀNH CHÍNH)

7. HỌC PHẦN

Thuộc tính	Vai trò / Ghi chú
MaHP	Khóa chính (PK) — đề quy định "mã học phần duy nhất"
TenHP	Thuộc tính mô tả
SoTinChi	Thuộc tính mô tả — ràng buộc $1 \leq \text{giá trị} \leq 6$

8. LỚP HỌC PHẦN

Thuộc tính	Vai trò / Ghi chú
MaLHP	Khóa chính (PK) — đề quy định "mã duy nhất"
MaHP	Khóa ngoại (FK → HỌC PHẦN)
HocKy	Thuộc tính mô tả
NamHoc	Thuộc tính mô tả
PhongHoc	Thuộc tính mô tả
SiSoToiDa	Thuộc tính mô tả
MaGVPhuTrach	Khóa ngoại (FK → GIẢNG VIÊN)

B. Các thực thể liên kết (associative entities)

9. GỒM (associative entity của quan hệ Ngành – Học phần)

Thuộc tính	Vai trò / Ghi chú
MaNganh	Khóa chính (PK, thành phần) — FK → NGÀNH
MaHP	Khóa chính (PK, thành phần) — FK → HỌC PHẦN
LoaiHP	Thuộc tính riêng của quan hệ — nhận giá trị "Bắt buộc" / "Tự chọn"

10. TIỀN QUYẾT (associative entity tự thân của Học phần)

Thuộc tính	Vai trò / Ghi chú
MaHP	Khóa chính (PK, thành phần) — FK → HỌC PHẦN
MaHP_TienQuyét	Khóa chính (PK, thành phần) — FK → HỌC PHẦN
(ràng buộc)	CHECK: MaHP ≠ MaHP_TienQuyét; không cho phép tạo chu trình

11. ĐĂNG KÝ (associative entity của quan hệ Sinh viên – Lớp học phần)

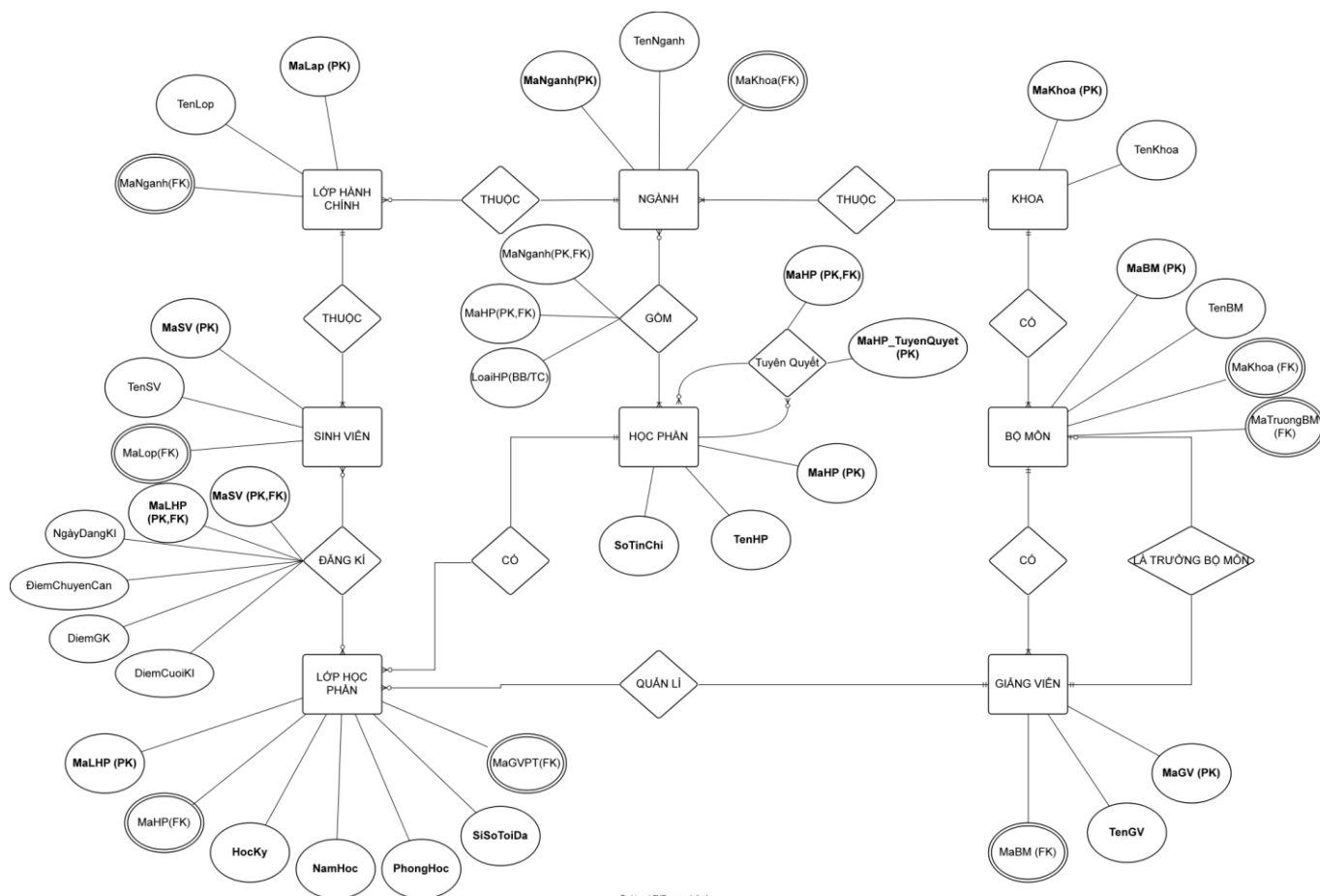
Thuộc tính	Vai trò / Ghi chú
MaSV	Khóa chính (PK, thành phần) — FK → SINH VIÊN
MaLHP	Khóa chính (PK, thành phần) — FK → LỚP HỌC PHẦN
NgayDangKy	Thuộc tính riêng của quan hệ
DiemChuyenCan	Thuộc tính riêng — cho phép NULL
DiemGiuaKy	Thuộc tính riêng — cho phép NULL
DiemCuoiKy	Thuộc tính riêng — cho phép NULL

CÂU 2. XÁC ĐỊNH MỐI QUAN HỆ, LỰC LƯỢNG QUAN HỆ VÀ TÍNH BẮT BUỘC/TÙY CHỌN

Ký hiệu (min, max) thể hiện số lần tối thiểu và tối đa một thực thể ở một phía tham gia vào quan hệ. "Bắt buộc" (min = 1) nghĩa là mọi bản ghi của thực thể đó luôn phải tham gia quan hệ; "tùy chọn" (min = 0) nghĩa là có thể tồn tại bản ghi không tham gia.

Cặp thực thể	Tên quan hệ	Lực lượng	Tham gia phía 1	Tham gia phía 2
KHOA — BỘ MÔN	THUỘC	1 : N	KHOA: (1,1) bắt buộc	BỘ MÔN: (1,n) bắt buộc
BỘ MÔN — GIẢNG VIÊN	CÓ	1 : N	BỘ MÔN: (1,1) bắt buộc	GIẢNG VIÊN: (1,n) bắt buộc
BỘ MÔN — GIẢNG VIÊN	LÀ TRƯỞNG BM	1 : 1	BỘ MÔN: (0,1) tùy chọn	GIẢNG VIÊN: (1,1) bắt buộc
NGÀNH — KHOA*	THUỘC	1 : N	KHOA: (1,1) bắt buộc	NGÀNH: (1,n) bắt buộc
LỚP HÀNH CHÍNH — NGÀNH	THUỘC	1 : N	NGÀNH: (1,1) bắt buộc	LỚP HÀNH CHÍNH: (0,n) tùy chọn
SINH VIÊN — LỚP HÀNH CHÍNH	THUỘC	1 : N	LỚP HÀNH CHÍNH: (1,1) bắt buộc	SINH VIÊN: (1,n) bắt buộc
NGÀNH — HỌC PHẦN	GỒM	M : N	NGÀNH: (0,n) tùy chọn	HỌC PHẦN: (1,n) bắt buộc
HỌC PHẦN — HỌC PHẦN	TIỀN QUYẾT (tự thân)	M : N	Vai trò 1: (0,n) tùy chọn	Vai trò 2: (0,n) tùy chọn
HỌC PHẦN — LỚP HỌC PHẦN	CÓ	1 : N	HỌC PHẦN: (1,1) bắt buộc	LỚP HỌC PHẦN: (0,n) tùy chọn
GIẢNG VIÊN — LỚP HỌC PHẦN	QUẢN LÝ	1 : N	GIẢNG VIÊN: (1,1) bắt buộc	LỚP HỌC PHẦN: (0,n) tùy chọn
SINH VIÊN — LỚP HỌC PHẦN	ĐĂNG KÝ	M : N	SINH VIÊN: (0,n) tùy chọn	LỚP HỌC PHẦN: (0,n) tùy chọn

Câu 3. Vẽ ERD hoàn chỉnh. ERD phải thể hiện tối thiểu: KHOA, BỘ MÔN, GIẢNG VIÊN, NGÀNH, LỚP HÀNH CHÍNH, SINH VIÊN, HỌC PHẦN, CHƯƠNG TRÌNH ĐÀO TẠO, LỚP HỌC PHẦN, ĐĂNG KÝ và quan hệ TIỀN QUYẾT.



Câu 4:

Câu 5:

IV. PHẦN B – RÀNG BUỘC TOÀN VỆN

STT	Ràng buộc	Cách kiểm soát	Công cụ
1	Số tín chỉ từ 1–6	Giới hạn giá trị cột SoTinChi	CHECK
2	Điểm từ 0–10 hoặc NULL	Kiểm tra từng cột điểm	CHECK
3	Sĩ số tối đa > 0	Kiểm tra SiSoToiDa	CHECK
4	SV không đăng ký cùng LHP 2 lần	Không cho trùng cặp (MaSV, MaLHP)	PRIMARY KEY / UNIQUE
5	Trưởng BM phải thuộc BM	Kiểm tra GV trưởng thuộc đúng BM	FK ghép / Trigger
6	LHP phải thuộc một HP	MaHP bắt buộc và phải tồn tại	NOT NULL + FK
7	LHP có đúng một GV chính	MaGV bắt buộc và phải tồn tại	NOT NULL + FK

8	Không đăng ký khi lớp đầy	Đếm đăng ký trước khi thêm	Trigger / Stored Procedure
9	HP không tiên quyết chính nó	MaHP $\neq$ MaHPTienQuyét	CHECK
10	Không có chu trình tiên quyết	Kiểm tra quan hệ nhiều cấp	Recursive CTE / Trigger
11	Ngành–HP không trùng	Không cho trùng (MaNganh, MaHP)	PK / UNIQUE
12	Loại HP chỉ có 2 giá trị	Chỉ cho BẮT BUỘC, TỰ CHỌN	CHECK
13	Ngày đăng ký không NULL	Bắt buộc nhập ngày	NOT NULL
14	SV và LHP phải tồn tại	Kiểm tra khóa tham chiếu	FOREIGN KEY
15	GPA/Tín chỉ tích lũy không lưu dư thừa	Tính từ dữ liệu đăng ký khi cần	Query / VIEW

CÂU 5: file sql

Câu 6:file sql

Câu 7:file sql

CÂU 8. CỐ Ý VI PHẠM RÀNG BUỘC ĐỂ KIỂM THỬ

- Để kiểm tra các ràng buộc toàn vẹn đã cài đặt ở Câu 6, nhóm cố ý viết 9 lệnh INSERT/UPDATE vi phạm 9 ràng buộc khác nhau đã nêu ở Phần B. Mỗi lệnh được bọc trong khối TRY...CATCH của T-SQL: nếu SQL Server từ chối lệnh, thông báo lỗi được bắt lại trong CATCH và in ra màn hình thay vì làm dừng cả script, giúp chạy liên tục toàn bộ 9 lệnh thử nghiệm trong một lần Execute.

- Trong 9 lệnh, có 7 lệnh (8.1–8.7) bị SQL Server từ chối đúng như thiết kế, và 2 lệnh (8.8, 8.9) KHÔNG bị chặn — đây là 2 ràng buộc nghiệp vụ (B.5 và B.10) mà PRIMARY KEY/FOREIGN KEY/CHECK không thể cài đặt được vì cần so sánh dữ liệu giữa nhiều dòng/nhiều bảng hoặc duyệt quan hệ đệ quy. Hai ràng buộc này sẽ được xử lý bằng TRIGGER ở phần bài tập nâng cao (Câu 35); trong lần kiểm thử này, dữ liệu sai được hoàn tác (UPDATE/DELETE) ngay sau khi mình chứng để không ảnh hưởng các câu truy vấn khác.

Bảng tổng hợp:

STT	Ràng buộc bị vi phạm	Kết quả thực tế	Cơ chế xử lý
8.1	B.1 - Số tín chỉ 1–6 (CHECK)	Bị từ chối	CHECK constraint
8.2	B.2 - Điểm 0–10 (CHECK)	Bị từ chối	CHECK constraint
8.3	B.4 - Không đăng ký trùng 1 lớp (PRIMARY KEY)	Bị từ chối	PK constraint
8.4	B.14 - SV/LHP tham chiếu phải tồn tại (FOREIGN KEY)	Bị từ chối	FK constraint
8.5	B.13 - Ngày đăng ký không được trống (NOT NULL)	Bị từ chối	NOT NULL constraint

STT	Ràng buộc bị vi phạm	Kết quả thực tế	Cơ chế xử lý
8.6	B.9 - Học phần không là tiên quyết của chính nó (CHECK)	Bị từ chối	CHECK constraint
8.7	B.8 - Không vượt số tối đa	Bị từ chối	TRIGGER (INSTEAD OF INSERT)
8.8	B.5 - Trưởng bộ môn phải thuộc chính bộ môn đó	KHÔNG bị chặn	Chưa có cơ chế kiểm soát
8.9	B.10 - Quan hệ tiên quyết không được tạo chu trình	KHÔNG bị chặn	Chưa có cơ chế kiểm soát

8.1. Vi phạm B.1 — Số tín chỉ phải từ 1 đến 6 (CHECK)

Câu lệnh SQL:

```
INSERT INTO HOC_PHAN (MaHP, TenHP, SoTinChi)
VALUES ('HPX1', N'Học phần lỗi tín chỉ', 7);
```

Kết quả khi thực thi:

Bị từ chối — Msg 547: The INSERT statement conflicted with the CHECK constraint "CK_HOCPHAN_TINCHI".

Giải thích:

Cột SoTinChi có ràng buộc CHECK (SoTinChi BETWEEN 1 AND 6). Giá trị 7 nằm ngoài khoảng cho phép nên SQL Server từ chối ngay khi kiểm tra ràng buộc, không ghi dòng nào vào bảng HOC_PHAN.

8.2. Vi phạm B.2 — Điểm phải trong khoảng 0–10 (CHECK)

Câu lệnh SQL:

```
INSERT INTO DANG_KY (MaSV, MaLHP, NgayDangKy, DiemChuyenCan,
DiemGiuaKy, DiemCuoiKy)
VALUES ('SV01','LHP13','2025-08-20',5,5,15);
```

Kết quả khi thực thi:

Bị từ chối — Msg 547: The INSERT statement conflicted with the CHECK constraint "CK_DK_CUOIKY".

Giải thích:

DiemCuoiKy = 15 vi phạm CHECK (DiemCuoiKy BETWEEN 0 AND 10). Lớp LHP13 được chọn vì đang còn trống chỗ (chưa có đăng ký nào), để chắc chắn lỗi hiển thị đúng là do điểm sai, không bị trigger số ở B.8 chặn trước gây nhiễu kết quả.

8.3. Vi phạm B.4 / PRIMARY KEY — Không được đăng ký trùng 1 lớp học phần

Câu lệnh SQL:

```
-- SV16 đã đăng ký LHP12 từ Câu 7, đăng ký lại lần nữa
INSERT INTO DANG_KY (MaSV, MaLHP, NgayDangKy, DiemChuyenCan,
DiemGiuaKy, DiemCuoiKy)
VALUES ('SV16','LHP12','2024-08-25',9,9,9);
```

Kết quả khi thực thi:

Bị từ chối — Msg 2627: Violation of PRIMARY KEY constraint "PK_DANGKY". Cannot insert duplicate key.

Giải thích:

Khóa chính của DANG_KY là (MaSV, MaLHP). Cặp (SV16, LHP12) đã tồn tại từ dữ liệu Câu 7 nên không thể thêm lần nữa — đây chính là cách PRIMARY KEY tự động đảm bảo ràng buộc B.4. LHP12 được chọn vì lớp còn 1 chỗ trống, đảm bảo lỗi xảy ra do trùng khóa chứ không phải do vượt số.

8.4. Vi phạm B.14 / FOREIGN KEY — Sinh viên tham chiếu phải tồn tại

Câu lệnh SQL:

```
INSERT INTO DANG_KY (MaSV, MaLHP, NgayDangKy, DiemChuyenCan,
DiemGiuaKy, DiemCuoiKy)
VALUES ('SV99','LHP13','2025-08-20',8,8,8);
```

Kết quả khi thực thi:

Bị từ chối — Msg 547: The INSERT statement conflicted with the FOREIGN KEY constraint "FK_DK_SINHVIEN".

Giải thích:

Mã sinh viên 'SV99' không tồn tại trong bảng SINH_VIEN. Ràng buộc FOREIGN KEY đảm bảo mọi giá trị MaSV trong DANG_KY phải khớp với một dòng có thật ở SINH_VIEN.

8.5. Vi phạm B.13 / NOT NULL — Ngày đăng ký không được để trống

Câu lệnh SQL:

```
INSERT INTO DANG_KY (MaSV, MaLHP, NgayDangKy, DiemChuyenCan,
DiemGiuaKy, DiemCuoiKy)
VALUES ('SV04','LHP13', NULL, 8,8,8);
```

Kết quả khi thực thi:

Bị từ chối — Msg 515: Cannot insert the value NULL into column 'NgayDangKy'.

Giải thích:

Cột NgayDangKy được khai báo NOT NULL nên SQL Server chặn ngay việc ghi giá trị NULL vào cột này.

8.6. Vi phạm B.9 — Học phần không thể là tiên quyết của chính nó (CHECK)

Câu lệnh SQL:

```
INSERT INTO HOC_PHAN_TIEN_QUYET (MaHP, MaHP_TienQuyet)
VALUES ('HP01','HP01');
```

Kết quả khi thực thi:

Bị từ chối — Msg 547: The INSERT statement conflicted with the CHECK constraint "CK_HPTQ_KHACNHAU".

Giải thích:

Ràng buộc CHECK (MaHP <> MaHP_TienQuyet) không cho phép 2 cột này bằng nhau trên cùng 1 dòng, nên trường hợp một học phần tự làm tiên quyết của chính nó bị chặn ngay tại tầng CSDL.

8.7. Vi phạm B.8 — Không vượt số tối đa (xử lý bằng TRIGGER)

Câu lệnh SQL:

```
-- LHP06 đã đủ 3/3 sinh viên từ Câu 7
INSERT INTO DANG_KY (MaSV, MaLHP, NgayDangKy, DiemChuyenCan,
DiemGiuaKy, DiemCuoiKy)
VALUES ('SV07','LHP06','2025-01-15',8,8,8);
```

Kết quả khi thực thi:

Bị từ chối — lỗi tự định nghĩa qua RAISERROR: "Lớp học phần đã đủ số, không thể đăng ký thêm."

Giải thích:

Ràng buộc này không thể cài bằng CHECK vì CHECK không được phép SELECT COUNT(*) sang bảng khác. Trigger TRG_DANGKY_KIEMTRA_SISO (loại INSTEAD OF INSERT) đếm số sinh viên đã đăng ký lớp LHP06 (3) cộng với số dòng đang cố thêm (1) = 4, vượt SiSoToiDa = 3 nên trigger raise lỗi và không thực hiện INSERT.

8.8. Vi phạm B.5 — Trưởng bộ môn phải thuộc chính bộ môn đó (CHƯA bị chặn)

Câu lệnh SQL:

```
-- GV07 thuộc BM04, KHÔNG thuộc BM01
UPDATE BO_MON SET MaTruongBM = 'GV07' WHERE MaBM = 'BM01';
SELECT MaBM, MaTruongBM FROM BO_MON WHERE MaBM = 'BM01';
```

```
-- Hoàn tác ngay sau khi mình chứng
UPDATE BO_MON SET MaTruongBM = 'GV01' WHERE MaBM = 'BM01';
```

Kết quả khi thực thi:

KHÔNG bị chặn — lệnh UPDATE thực thi thành công, kết quả SELECT cho thấy BM01 bị gán trường bộ môn là GV07 dù GV07 thuộc BM04.

Giải thích:

FOREIGN KEY trên cột MaTruongBM chỉ kiểm tra giá trị đó có tồn tại trong bảng GIANG_VIEN hay không, chứ không so sánh MaBM của giảng viên được chọn với MaBM của chính bộ môn đang cập nhật. Đây là ràng buộc liên-cột-liên-bảng mà PK/FK/CHECK không biểu diễn được, cần một TRIGGER kiểm tra sau khi UPDATE/INSERT (sẽ cài đặt ở Câu 35). Dữ liệu đã được hoàn tác ngay trong cùng lệnh để không ảnh hưởng các câu truy vấn phía sau.

8.9. Vi phạm B.10 — Quan hệ tiên quyết không được tạo chu trình (CHƯA bị chặn)

Câu lệnh SQL:

```
-- Tạo cạnh HP01 -> HP12, khép thành chu trình
-- HP01 -> HP02 -> HP04 -> HP05 -> HP12 -> HP01
INSERT INTO HOC_PHAN_TIEN_QUYET (MaHP, MaHP_TienQuyet) VALUES
('HP01','HP12');
SELECT * FROM HOC_PHAN_TIEN_QUYET
WHERE MaHP='HP01' AND MaHP_TienQuyet='HP12';

-- Hoàn tác ngay sau khi mình chứng
DELETE FROM HOC_PHAN_TIEN_QUYET
WHERE MaHP='HP01' AND MaHP_TienQuyet='HP12';
```

Kết quả khi thực thi:

KHÔNG bị chặn — dòng dữ liệu được thêm thành công, tạo ra chu trình tiên quyết HP01 → HP02 → HP04 → HP05 → HP12 → HP01.

Giải thích:

PRIMARY KEY/CHECK chỉ kiểm tra được giá trị trong phạm vi 1 dòng đang ghi, không thể duyệt toàn bộ đồ thị quan hệ tiên quyết để phát hiện chu trình. Việc phát hiện chu trình đòi hỏi duyệt đệ quy (ví dụ bằng Recursive CTE) trong một TRIGGER, sẽ được cài đặt ở Câu 35. Dữ liệu đã được xóa ngay sau khi mình chứng.

PHẦN D – TRUY VẤN SQL

Nhóm 1 – JOIN, GROUP BY và truy vấn nền tảng (Câu 1–10)

Câu D1. Liệt kê sinh viên: mã SV, họ tên, lớp hành chính, ngành

Yêu cầu: Liệt kê sinh viên gồm mã sinh viên, họ tên, lớp hành chính và ngành.

Kỹ thuật sử dụng: INNER JOIN 3 bảng SINH_VIEN → LOP_HANH_CHINH → NGANH.

Câu lệnh SQL:

```
SELECT sv.MaSV, sv.TenSV, lhc.TenLop, n.TenNganh
FROM SINH_VIEN sv
JOIN LOP_HANH_CHINH lhc ON sv.MaLop = lhc.MaLop
JOIN NGANH n          ON lhc.MaNganh = n.MaNganh
ORDER BY sv.MaSV;
```

Giải thích:

Mỗi sinh viên thuộc đúng 1 lớp hành chính, mỗi lớp hành chính thuộc đúng 1 ngành, nên chuỗi JOIN 1-N liên tiếp là đủ để lấy ra đầy đủ thông tin mà không phát sinh trùng lặp dòng.

Câu D2. Liệt kê lớp học phần: mã lớp, tên học phần, học kỳ, năm học, GV phụ trách

Yêu cầu: Liệt kê các lớp học phần gồm mã lớp, tên học phần, học kỳ, năm học và giảng viên phụ trách.

Kỹ thuật sử dụng: INNER JOIN LOP_HOC_PHAN với HOC_PHAN và GIANG_VIEN.

Câu lệnh SQL:

```
SELECT lhp.MaLHP, hp.TenHP, lhp.HocKy, lhp.NamHoc, gv.TenGV
FROM LOP_HOC_PHAN lhp
JOIN HOC_PHAN hp    ON lhp.MaHP = hp.MaHP
JOIN GIANG_VIEN gv  ON lhp.MaGVPT = gv.MaGV
ORDER BY lhp.NamHoc, lhp.HocKy, lhp.MaLHP;
```

Giải thích:

Vì mỗi lớp học phần bắt buộc có đúng 1 học phần và đúng 1 giảng viên phụ trách chính (NOT NULL + FK), INNER JOIN không làm mất dữ liệu nào.

Câu D3. Đếm số sinh viên đăng ký từng lớp học phần, kể cả lớp chưa có đăng ký

Yêu cầu: Đếm số sinh viên đăng ký trong từng lớp học phần, kể cả lớp chưa có sinh viên đăng ký.

Kỹ thuật sử dụng: LEFT JOIN (giữ lại lớp không có đăng ký) + GROUP BY + COUNT trên cột của bảng bên phải.

Câu lệnh SQL:

```
SELECT lhp.MaLHP, lhp.MaHP, COUNT(dk.MaSV) AS SoSinhVienDangKy
FROM LOP_HOC_PHAN lhp
LEFT JOIN DANG_KY dk ON lhp.MaLHP = dk.MaLHP
GROUP BY lhp.MaLHP, lhp.MaHP
ORDER BY lhp.MaLHP;
```

Giải thích:

Nếu dùng INNER JOIN, lớp không có đăng ký nào (như LHP13) sẽ biến mất khỏi kết quả. LEFT JOIN giữ lại lớp đó với COUNT(dk.MaSV) = 0 vì COUNT bỏ qua giá trị NULL.

Câu D4. Tìm các lớp học phần đã đủ sĩ số

Yêu cầu: Tìm các lớp học phần đã đủ sĩ số.

Kỹ thuật sử dụng: JOIN + GROUP BY + HAVING so sánh số lượng đăng ký thực tế với SiSoToiDa.

Câu lệnh SQL:

```
SELECT lhp.MaLHP, lhp.SiSoToiDa, COUNT(dk.MaSV) AS SiSoHienTai
FROM LOP_HOC_PHAN lhp
JOIN DANG_KY dk ON lhp.MaLHP = dk.MaLHP
GROUP BY lhp.MaLHP, lhp.SiSoToiDa
HAVING COUNT(dk.MaSV) >= lhp.SiSoToiDa;
```

Giải thích:

HAVING lọc sau khi đã GROUP BY, cho phép so sánh giá trị tổng hợp (COUNT) với cột SiSoToiDa của chính lớp đó.

Câu D5. Tìm lớp học phần còn chỗ và số chỗ còn lại

Yêu cầu: Tìm các lớp học phần còn chỗ và cho biết chính xác số chỗ còn lại.

Kỹ thuật sử dụng: LEFT JOIN (không bỏ sót lớp chưa ai đăng ký) + HAVING trên biểu thức hiệu số.

Câu lệnh SQL:

```
SELECT lhp.MaLHP, lhp.SiSoToiDa,
       COUNT(dk.MaSV) AS SiSoHienTai,
       lhp.SiSoToiDa - COUNT(dk.MaSV) AS SoChoConLai
FROM LOP_HOC_PHAN lhp
LEFT JOIN DANG_KY dk ON lhp.MaLHP = dk.MaLHP
GROUP BY lhp.MaLHP, lhp.SiSoToiDa
HAVING lhp.SiSoToiDa - COUNT(dk.MaSV) > 0
ORDER BY lhp.MaLHP;
```

Giải thích:

Dùng LEFT JOIN để các lớp chưa có đăng ký (COUNT = 0) vẫn xuất hiện với số chỗ còn lại bằng đúng số tối đa.

Câu D6. Tính điểm tổng kết của từng lượt đăng ký

Yêu cầu: Tính điểm tổng kết của từng lượt đăng ký theo công thức đã cho.

Kỹ thuật sử dụng: CASE WHEN kiểm tra đủ 3 cột điểm trước khi tính, tránh phép toán trên NULL.

Câu lệnh SQL:

```
SELECT dk.MaSV, dk.MaLHP,
CASE
    WHEN dk.DiemChuyenCan IS NULL OR dk.DiemGiuaKy IS NULL OR
    dk.DiemCuoiKy IS NULL THEN NULL
    ELSE ROUND(0.1*dk.DiemChuyenCan + 0.3*dk.DiemGiuaKy +
    0.6*dk.DiemCuoiKy, 2)
END AS DiemTongKet
FROM DANG_KY dk
ORDER BY dk.MaSV, dk.MaLHP;
```

Giải thích:

Nếu điểm chưa nhập đủ (còn NULL) thì kết quả tính toán trong SQL Server tự động là NULL; CASE WHEN chỉ nhằm thể hiện tường minh và cho phép làm tròn 2 chữ số bằng ROUND.

Câu D7. Tìm sinh viên có ít nhất một lần trượt học phần

Yêu cầu: Tìm các sinh viên có ít nhất một lần trượt học phần.

Kỹ thuật sử dụng: JOIN + WHERE lọc điều kiện đủ điểm và điểm tổng kết < 4 + DISTINCT.

Câu lệnh SQL:

```
SELECT DISTINCT sv.MaSV, sv.TenSV
FROM SINH_VIEN sv
JOIN DANG_KY dk ON sv.MaSV = dk.MaSV
WHERE dk.DiemChuyenCan IS NOT NULL AND dk.DiemGiuaKy IS NOT NULL
AND dk.DiemCuoiKy IS NOT NULL
AND (0.1*dk.DiemChuyenCan + 0.3*dk.DiemGiuaKy + 0.6*dk.DiemCuoiKy) < 4;
```

Giải thích:

Chỉ xét các lượt đăng ký đã có đủ 3 thành phần điểm (đã có kết quả cuối cùng); DISTINCT để mỗi sinh viên chỉ xuất hiện 1 lần dù trượt nhiều học phần.

Câu D8. SV đã có kết quả nhưng chưa từng trượt

Yêu cầu: Tìm các sinh viên đã có kết quả học tập nhưng chưa từng trượt bất kỳ học phần nào.

Kỹ thuật sử dụng: EXISTS (đã có ít nhất 1 lượt đủ điểm) kết hợp NOT EXISTS (không có lượt nào trượt).

Câu lệnh SQL:

```
SELECT sv.MaSV, sv.TenSV
FROM SINH_VIEN sv
WHERE EXISTS (
    SELECT 1 FROM DANG_KY dk
    WHERE dk.MaSV = sv.MaSV
    AND dk.DiemChuyenCan IS NOT NULL AND dk.DiemGiuaKy IS NOT NULL
    AND dk.DiemCuoiKy IS NOT NULL
) AND NOT EXISTS (
    SELECT 1 FROM DANG_KY dk
    WHERE dk.MaSV = sv.MaSV
    AND dk.DiemChuyenCan IS NOT NULL AND dk.DiemGiuaKy IS NOT NULL
    AND dk.DiemCuoiKy IS NOT NULL
    AND (0.1*dk.DiemChuyenCan + 0.3*dk.DiemGiuaKy + 0.6*dk.DiemCuoiKy) < 4
);
```

Giải thích:

Kết hợp 2 điều kiện EXISTS/NOT EXISTS để loại cả 2 trường hợp không mong muốn: sinh viên chưa có điểm nào, và sinh viên có ít nhất 1 lần trượt.

Câu D9. SV học cùng một học phần từ hai lần trở lên

Yêu cầu: Tìm các sinh viên đã học cùng một học phần từ hai lần trở lên.

Kỹ thuật sử dụng: JOIN quy đổi lớp học phần về học phần gốc + GROUP BY (SV, HP) + HAVING COUNT >= 2.

Câu lệnh SQL:

```
SELECT dk.MaSV, lhp.MaHP, COUNT(*) AS SoLanHoc
FROM DANG_KY dk
JOIN LOP_HOC_PHAN lhp ON dk.MaLHP = lhp.MaLHP
GROUP BY dk.MaSV, lhp.MaHP
HAVING COUNT(*) >= 2;
```

Giải thích:

Một sinh viên có thể đăng ký nhiều lớp học phần khác nhau nhưng cùng 1 mã học phần (học lại); nhóm theo cặp (MaSV, MaHP) rồi đếm số dòng sẽ phát hiện các trường hợp học lại.

Câu D10. Học phần có tổng số lượt đăng ký lớn nhất

Yêu cầu: Tìm học phần có tổng số lượt sinh viên đăng ký lớn nhất.

Kỹ thuật sử dụng: GROUP BY + TOP 1 WITH TIES để xử lý trường hợp đồng hạng.

Câu lệnh SQL:

```
SELECT TOP 1 WITH TIES lhp.MaHP, hp.TenHP, COUNT(*) AS TongLuotDangKy
FROM DANG_KY dk
JOIN LOP_HOC_PHAN lhp ON dk.MaLHP = lhp.MaLHP
JOIN HOC_PHAN hp ON lhp.MaHP = hp.MaHP
GROUP BY lhp.MaHP, hp.TenHP
ORDER BY COUNT(*) DESC;
```

Giải thích:

TOP 1 WITH TIES kết hợp ORDER BY sẽ trả về tất cả các học phần có cùng giá trị lớn nhất, thay vì chỉ 1 dòng bất kỳ như TOP 1 thông thường.

Nhóm 2 – GROUP BY, Subquery và NOT EXISTS (Câu 11–20)

Câu D11. Giảng viên phụ trách nhiều lớp học phần nhất

Yêu cầu: Tìm giảng viên phụ trách nhiều lớp học phần nhất.

Kỹ thuật sử dụng: GROUP BY + TOP 1 WITH TIES.

Câu lệnh SQL:

```
SELECT TOP 1 WITH TIES gv.MaGV, gv.TenGV, COUNT(*) AS SoLopPhuTrach
FROM LOP_HOC_PHAN lhp
JOIN GIANG_VIEN gv ON lhp.MaGVPT = gv.MaGV
GROUP BY gv.MaGV, gv.TenGV
ORDER BY COUNT(*) DESC;
```

Giải thích:

Tương tự câu D10, gom nhóm theo giảng viên phụ trách chính rồi lấy giá trị lớn nhất, có xử lý đồng hạng.

Câu D12. Sinh viên có GPA tích lũy cao nhất toàn trường

Yêu cầu: Tìm sinh viên có GPA tích lũy cao nhất toàn trường.

Kỹ thuật sử dụng: CTE 2 tầng: DiemDayDu (chỉ lấy lượt đã đủ điểm) → BestAttempt (MAX điểm tổng kết theo từng học phần) → tính GPA có trọng số theo tín chỉ.

Câu lệnh SQL:

```
WITH DiemDayDu AS (
    SELECT dk.MaSV, lhp.MaHP, hp.SoTinChi,
           (0.1*dk.DiemChuyenCan + 0.3*dk.DiemGiuaKy + 0.6*dk.DiemCuoiKy) AS
    DiemTongKet
    FROM DANG_KY dk
```

```

JOIN LOP_HOC_PHAN lhp ON dk.MaLHP = lhp.MaLHP
JOIN HOC_PHAN hp ON lhp.MaHP = hp.MaHP
WHERE dk.DiemChuyenCan IS NOT NULL AND dk.DiemGiuaKy IS NOT NULL
AND dk.DiemCuoiKy IS NOT NULL
),
BestAttempt AS (
    SELECT MaSV, MaHP, SoTinChi, MAX(DiemTongKet) AS
    DiemTongKetCaoNhat
    FROM DiemDayDu
    GROUP BY MaSV, MaHP, SoTinChi
),
GPA_SV AS (
    SELECT MaSV, SUM(DiemTongKetCaoNhat*SoTinChi) / SUM(SoTinChi) AS
    GPA
    FROM BestAttempt GROUP BY MaSV
)
SELECT TOP 1 WITH TIES sv.MaSV, sv.TenSV, g.GPA
FROM GPA_SV g JOIN SINH_VIEN sv ON sv.MaSV = g.MaSV
ORDER BY g.GPA DESC;

```

Giải thích:

Đây là mẫu CTE dùng lại xuyên suốt các câu về GPA/tín chỉ: bước 1 lọc các lượt đăng ký đã có đủ 3 điểm; bước 2 với mỗi học phần chỉ giữ lại lần có điểm tổng kết CAO NHẤT (đúng quy ước 'học lại chỉ lấy điểm cao nhất'); bước 3 tính GPA có trọng số theo số tín chỉ.

Câu D13. SV có GPA cao nhất trong từng lớp hành chính (xử lý đồng hạng)

Yêu cầu: Tìm sinh viên có GPA cao nhất trong từng lớp hành chính; nếu đồng hạng phải trả về tất cả.

Kỹ thuật sử dụng: CTE tính GPA (giống D12) + RANK() OVER (PARTITION BY lớp hành chính ORDER BY GPA DESC).

Câu lệnh SQL:

```

-- (dùng lại CTE DiemDayDu, BestAttempt, GPA_SV như câu D12)
XepHang AS (
    SELECT sv.MaSV, sv.TenSV, sv.MaLop, g.GPA,
           RANK() OVER (PARTITION BY sv.MaLop ORDER BY g.GPA DESC) AS
    Hang
    FROM GPA_SV g JOIN SINH_VIEN sv ON sv.MaSV = g.MaSV
)
SELECT MaLop, MaSV, TenSV, GPA
FROM XepHang
WHERE Hang = 1

```

ORDER BY MaLop;

Giải thích:

PARTITION BY MaLop khiến RANK() được tính riêng cho từng lớp hành chính; RANK (khác ROW_NUMBER) gán cùng thứ hạng 1 cho các GPA bằng nhau, đáp ứng đúng yêu cầu trả về tất cả trường hợp đồng hạng.

Câu D14. SV có điểm tổng kết cao hơn điểm trung bình của lớp học phần

Yêu cầu: Với mỗi lớp học phần, tìm sinh viên có điểm tổng kết cao hơn điểm trung bình của chính lớp đó.

Kỹ thuật sử dụng: Subquery tương quan (correlated subquery) tính AVG() theo từng MaLHP.

Câu lệnh SQL:

```
SELECT dk.MaLHP, dk.MaSV,
       (0.1*dk.DiemChuyenCan + 0.3*dk.DiemGiuaKy + 0.6*dk.DiemCuoiKy) AS
       DiemTongKet
FROM DANG_KY dk
WHERE dk.DiemChuyenCan IS NOT NULL AND dk.DiemGiuaKy IS NOT NULL
      AND dk.DiemCuoiKy IS NOT NULL
      AND (0.1*dk.DiemChuyenCan + 0.3*dk.DiemGiuaKy + 0.6*dk.DiemCuoiKy) >
          (SELECT AVG(0.1*dk2.DiemChuyenCan + 0.3*dk2.DiemGiuaKy +
                     0.6*dk2.DiemCuoiKy)
           FROM DANG_KY dk2
           WHERE dk2.MaLHP = dk.MaLHP
                 AND dk2.DiemChuyenCan IS NOT NULL AND dk2.DiemGiuaKy IS NOT
                 NULL AND dk2.DiemCuoiKy IS NOT NULL);
```

Giải thích:

Subquery bên trong tham chiếu tới dk.MaLHP của dòng đang xét ở truy vấn ngoài (dk2.MaLHP = dk.MaLHP), nên với mỗi dòng, SQL Server sẽ tính lại điểm trung bình riêng cho đúng lớp học phần chứa dòng đó — đây là đặc trưng của subquery tương quan.

Câu D15. SV đạt điểm tổng kết cao nhất trong từng lớp học phần (đồng điểm)

Yêu cầu: Tìm sinh viên đạt điểm tổng kết cao nhất trong từng lớp học phần; phải xử lý trường hợp đồng điểm.

Kỹ thuật sử dụng: RANK() OVER (PARTITION BY MaLHP ORDER BY DiemTongKet DESC).

Câu lệnh SQL:

```
WITH DiemLop AS (
  SELECT dk.MaLHP, dk.MaSV,
```

```
(0.1*dk.DiemChuyenCan + 0.3*dk.DiemGiuaKy + 0.6*dk.DiemCuoiKy) AS
DiemTongKet
FROM DANG_KY dk
WHERE dk.DiemChuyenCan IS NOT NULL AND dk.DiemGiuaKy IS NOT NULL
AND dk.DiemCuoiKy IS NOT NULL
),
XepHang AS (
    SELECT *, RANK() OVER (PARTITION BY MaLHP ORDER BY DiemTongKet
DESC) AS Hang
    FROM DiemLop
)
SELECT MaLHP, MaSV, DiemTongKet
FROM XepHang WHERE Hang = 1
ORDER BY MaLHP;
```

Giải thích:

Giống D13 nhưng PARTITION BY theo lớp học phần thay vì lớp hành chính; dùng RANK() để mọi sinh viên cùng đạt điểm cao nhất trong lớp đều được liệt kê.

Câu D16. SV đăng ký tổng tín chỉ lớn nhất trong 1 học kỳ/năm học

Yêu cầu: Với một học kỳ và năm học được chỉ định, tìm sinh viên đăng ký tổng số tín chỉ lớn nhất.

Kỹ thuật sử dụng: Biến (DECLARE) + JOIN + GROUP BY + TOP 1 WITH TIES.

Câu lệnh SQL:

```
DECLARE @HocKy INT = 1;
DECLARE @NamHoc VARCHAR(9) = '2024-2025';

SELECT TOP 1 WITH TIES dk.MaSV, SUM(hp.SoTinChi) AS TongTinChiDangKy
FROM DANG_KY dk
JOIN LOP_HOC_PHAN lhp ON dk.MaLHP = lhp.MaLHP
JOIN HOC_PHAN hp ON lhp.MaHP = hp.MaHP
WHERE lhp.HocKy = @HocKy AND lhp.NamHoc = @NamHoc
GROUP BY dk.MaSV
ORDER BY SUM(hp.SoTinChi) DESC;
```

Giải thích:

Lọc theo học kỳ/năm học được truyền vào bằng biến trước, sau đó cộng dồn số tín chỉ các học phần sinh viên đã đăng ký trong đúng kỳ đó.

Câu D17. SV đăng ký tất cả học phần có mở lớp trong 1 học kỳ/năm học

Yêu cầu: Tìm sinh viên đăng ký tất cả các học phần có mở lớp trong một học kỳ/năm học được chỉ định (1 học phần mở nhiều lớp thì chỉ cần đăng ký 1 lớp).

Kỹ thuật sử dụng: NOT EXISTS lồng nhau — phủ định của 'tồn tại 1 HP mà SV chưa đăng ký lớp nào của HP đó trong kỳ'.

Câu lệnh SQL:

```
DECLARE @HocKy2 INT = 1;
DECLARE @NamHoc2 VARCHAR(9) = '2025-2026';

SELECT sv.MaSV, sv.TenSV
FROM SINH_VIEN sv
WHERE NOT EXISTS (
    SELECT 1 FROM LOP_HOC_PHAN lhp
    WHERE lhp.HocKy = @HocKy2 AND lhp.NamHoc = @NamHoc2
    AND NOT EXISTS (
        SELECT 1 FROM DANG_KY dk
        JOIN LOP_HOC_PHAN lhp2 ON dk.MaLHP = lhp2.MaLHP
        WHERE dk.MaSV = sv.MaSV AND lhp2.MaHP = lhp.MaHP
        AND lhp2.HocKy = @HocKy2 AND lhp2.NamHoc = @NamHoc2
    )
);
```

Giải thích:

Đây là kỹ thuật 'lượng từ với mọi' (FOR ALL) kinh điển bằng NOT EXISTS lồng đôi: SV thỏa mãn khi KHÔNG tồn tại bất kỳ học phần mở trong kỳ mà SV chưa đăng ký lớp nào của nó.

Câu D18. Học phần đã mở lớp nhưng chưa từng có SV đăng ký

Yêu cầu: Tìm các học phần đã từng được mở lớp nhưng chưa từng có sinh viên đăng ký.

Kỹ thuật sử dụng: EXISTS (đã có lớp mở) kết hợp NOT EXISTS (không có đăng ký ở bất kỳ lớp nào của HP).

Câu lệnh SQL:

```
SELECT hp.MaHP, hp.TenHP
FROM HOC_PHAN hp
WHERE EXISTS (SELECT 1 FROM LOP_HOC_PHAN lhp WHERE lhp.MaHP = hp.MaHP)
AND NOT EXISTS (
    SELECT 1 FROM DANG_KY dk
    JOIN LOP_HOC_PHAN lhp ON dk.MaLHP = lhp.MaLHP
    WHERE lhp.MaHP = hp.MaHP
);
```

Giải thích:

Dữ liệu kiểm thử đã chủ động tạo học phần HP12 với lớp LHP13 không có đăng ký nào để phục vụ đúng câu truy vấn này.

Câu D19. Giảng viên chưa từng được phân công phụ trách lớp học phần

Yêu cầu: Tìm giảng viên chưa từng được phân công phụ trách lớp học phần nào.

Kỹ thuật sử dụng: NOT EXISTS.

Câu lệnh SQL:

```
SELECT gv.MaGV, gv.TenGV
FROM GIANG_VIEN gv
WHERE NOT EXISTS (SELECT 1 FROM LOP_HOC_PHAN lhp WHERE
lhp.MaGVPT = gv.MaGV);
```

Giải thích:

Giảng viên GV08 được cố tình không gán làm giảng viên phụ trách bất kỳ lớp nào trong dữ liệu kiểm thử để câu truy vấn này luôn trả về ít nhất 1 dòng.

Câu D20. Học phần không yêu cầu tiên quyết trực tiếp nào

Yêu cầu: Tìm học phần không yêu cầu bất kỳ học phần tiên quyết trực tiếp nào.

Kỹ thuật sử dụng: NOT EXISTS trên bảng HOC_PHAN_TIEN_QUYET.

Câu lệnh SQL:

```
SELECT hp.MaHP, hp.TenHP
FROM HOC_PHAN hp
WHERE NOT EXISTS (SELECT 1 FROM HOC_PHAN_TIEN_QUYET t WHERE
t.MaHP = hp.MaHP);
```

Giải thích:

Một học phần không có tiên quyết là học phần không xuất hiện ở cột MaHP (về 'học phần đang xét') của bảng quan hệ tiên quyết.

Nhóm 3 – Truy vấn nâng cao (Câu 21–30)

Câu D21. Học phần có ít nhất 2 học phần tiên quyết trực tiếp

Yêu cầu: Tìm học phần có ít nhất hai học phần tiên quyết trực tiếp.

Kỹ thuật sử dụng: GROUP BY + HAVING COUNT >= 2.

Câu lệnh SQL:

```
SELECT hp.MaHP, hp.TenHP, COUNT(*) AS SoTienQuyetTrucTiep
FROM HOC_PHAN_TIEN_QUYET t
JOIN HOC_PHAN hp ON hp.MaHP = t.MaHP
```

```
GROUP BY hp.MaHP, hp.TenHP
HAVING COUNT(*) >= 2;
```

Giải thích:

Dữ liệu kiểm thử thiết kế sẵn HP05 có 2 tiên quyết trực tiếp là HP03 và HP04 để phục vụ đúng câu truy vấn này.

Câu D22. Toàn bộ tiên quyết trực tiếp và gián tiếp của 1 học phần (kèm Level)

Yêu cầu: Với mã học phần @MaHP, tìm toàn bộ học phần tiên quyết trực tiếp và gián tiếp, đồng thời hiển thị mức (Level) của quan hệ tiên quyết.

Kỹ thuật sử dụng: Recursive CTE: Anchor là tiên quyết trực tiếp (Level 1), phần đệ quy nối thêm 1 mức mỗi vòng lặp.

Câu lệnh SQL:

```
DECLARE @MaHP VARCHAR(10) = 'HP12';

WITH TienQuyet AS (
    SELECT MaHP_TienQuyet AS MaHP_KetQua, 1 AS Level_
    FROM HOC_PHAN_TIEN QUYET
    WHERE MaHP = @MaHP

    UNION ALL

    SELECT t.MaHP_TienQuyet, tq.Level_ + 1
    FROM HOC_PHAN_TIEN QUYET t
    JOIN TienQuyet tq ON t.MaHP = tq.MaHP_KetQua
)
SELECT tq.MaHP_KetQua AS MaHP, hp.TenHP, MIN(tq.Level_) AS Level_
FROM TienQuyet tq
JOIN HOC_PHAN hp ON hp.MaHP = tq.MaHP_KetQua
GROUP BY tq.MaHP_KetQua, hp.TenHP
ORDER BY Level_;
```

Giải thích:

Chuỗi tiên quyết kiểm thử HP01→HP02→HP04→HP05→HP12 dài 4 mức cho phép kiểm tra Recursive CTE hoạt động đúng qua nhiều tầng đệ quy. GROUP BY + MIN(Level_) dùng để xử lý trường hợp 1 học phần có thể đạt tới bằng nhiều đường đi với độ dài khác nhau, khi đó lấy mức ngắn nhất.

Câu D23. HP là tiên quyết trực tiếp của nhiều học phần khác nhất

Yêu cầu: Tìm học phần đang là điều kiện tiên quyết trực tiếp của nhiều học phần khác nhất.

Kỹ thuật sử dụng: GROUP BY theo cột MaHP_TienQuyet + TOP 1 WITH TIES.

Câu lệnh SQL:

```
SELECT TOP 1 WITH TIES hp.MaHP, hp.TenHP, COUNT(*) AS
SoHocPhanPhuThuoc
FROM HOC_PHAN_TIEN_QUYET t
JOIN HOC_PHAN hp ON hp.MaHP = t.MaHP_TienQuyet
GROUP BY hp.MaHP, hp.TenHP
ORDER BY COUNT(*) DESC;
```

Giải thích:

Ngược chiều với câu D21: ở đây nhóm theo cột MaHP_TienQuyet (về 'là tiên quyết của') để đếm xem học phần nào được nhiều học phần khác phụ thuộc vào nhất.

Câu D24. Xếp hạng SV theo GPA tích lũy trong từng ngành bằng DENSE_RANK()

Yêu cầu: Xếp hạng sinh viên theo GPA tích lũy trong từng ngành bằng DENSE_RANK().

Kỹ thuật sử dụng: CTE tính GPA (như D12) + DENSE_RANK() OVER (PARTITION BY ngành ORDER BY GPA DESC).

Câu lệnh SQL:

```
-- (dùng lại CTE DiemDayDu, BestAttempt, GPA_SV như câu D12)
SELECT n.MaNganh, n.TenNganh, sv.MaSV, sv.TenSV, g.GPA,
       DENSE_RANK() OVER (PARTITION BY n.MaNganh ORDER BY g.GPA
DESC) AS XepHang
FROM GPA_SV g
JOIN SINH_VIEN sv ON sv.MaSV = g.MaSV
JOIN LOP_HANH_CHINH lhc ON sv.MaLop = lhc.MaLop
JOIN NGANH n ON lhc.MaNganh = n.MaNganh
ORDER BY n.MaNganh, XepHang;
```

Giải thích:

DENSE_RANK() khác RANK() ở chỗ không bỏ số thứ hạng khi có đồng hạng (ví dụ 2 SV cùng hạng 1 thì SV kế tiếp là hạng 2 chứ không phải hạng 3), đúng như đề bài yêu cầu dùng DENSE_RANK().

Câu D25. Top 3 SV có GPA cao nhất mỗi ngành (xử lý đồng hạng)

Yêu cầu: Tìm top 3 sinh viên có GPA cao nhất của mỗi ngành; xử lý rõ trường hợp đồng hạng.

Kỹ thuật sử dụng: CTE chứa DENSE_RANK() (như D24), sau đó lọc WHERE Hang <= 3 ở truy vấn ngoài.

Câu lệnh SQL:

```

WITH XepHang AS (
    SELECT n.MaNganh, n.TenNganh, sv.MaSV, sv.TenSV, g.GPA,
           DENSE_RANK() OVER (PARTITION BY n.MaNganh ORDER BY g.GPA
DESC) AS Hang
    FROM GPA_SV g
    JOIN SINH_VIEN sv      ON sv.MaSV = g.MaSV
    JOIN LOP_HANH_CHINH lhc ON sv.MaLop = lhc.MaLop
    JOIN NGANH n           ON lhc.MaNganh = n.MaNganh
)
SELECT MaNganh, TenNganh, MaSV, TenSV, GPA, Hang
FROM XepHang
WHERE Hang <= 3
ORDER BY MaNganh, Hang;

```

Giải thích:

Không thể lọc trực tiếp WHERE Hang <= 3 trong cùng 1 câu SELECT có window function, nên phải bọc trong CTE rồi lọc ở truy vấn ngoài. Dùng DENSE_RANK() thay vì ROW_NUMBER() để nếu có 4 SV cùng hạng 3 thì cả 4 đều được liệt kê thay vì bị cắt bớt.

Câu D26. Lớp học phần có tỷ lệ trượt cao nhất (trong các lớp đã có điểm đầy đủ)

Yêu cầu: Trong các lớp đã có điểm đầy đủ, tìm lớp học phần có tỷ lệ lượt sinh viên trượt cao nhất. Tỷ lệ trượt = số lượt TK<4 / tổng số lượt đã có đủ 3 thành phần điểm.

Kỹ thuật sử dụng: CTE thống kê theo lớp (tổng lượt, lượt đủ điểm, lượt trượt) + lọc lớp có 100% lượt đã đủ điểm + TOP 1 WITH TIES theo tỷ lệ.

Câu lệnh SQL:

```

WITH ThongKeLop AS (
    SELECT lhp.MaLHP,
           COUNT(*) AS TongLuotDangKy,
           SUM(CASE WHEN dk.DiemChuyenCan IS NOT NULL AND dk.DiemGiuaKy
IS NOT NULL AND dk.DiemCuoiKy IS NOT NULL
                THEN 1 ELSE 0 END) AS SoLuotDuDiem,
           SUM(CASE WHEN dk.DiemChuyenCan IS NOT NULL AND dk.DiemGiuaKy
IS NOT NULL AND dk.DiemCuoiKy IS NOT NULL
                AND
(0.1*dk.DiemChuyenCan+0.3*dk.DiemGiuaKy+0.6*dk.DiemCuoiKy) < 4
                THEN 1 ELSE 0 END) AS SoLuotTruot
    FROM LOP_HOC_PHAN lhp
    JOIN DANG_KY dk ON lhp.MaLHP = dk.MaLHP
    GROUP BY lhp.MaLHP
)
SELECT TOP 1 WITH TIES MaLHP, SoLuotTruot, SoLuotDuDiem,

```

```
CAST(SoLuotTruot AS FLOAT) / SoLuotDuDiem AS TyLeTruot
FROM ThongKeLop
WHERE TongLuotDangKy = SoLuotDuDiem AND SoLuotDuDiem > 0
ORDER BY TyLeTruot DESC;
```

Giải thích:

Điều kiện WHERE TongLuotDangKy = SoLuotDuDiem đảm bảo chỉ xét các lớp mà TẤT CẢ lượt đăng ký đều đã có đủ 3 cột điểm (đúng nghĩa 'lớp đã có điểm đầy đủ'); CAST sang FLOAT để tránh chia số nguyên bị làm tròn về 0.

Câu D27. Điểm tổng kết trung bình theo từng giảng viên phụ trách

Yêu cầu: Tính điểm tổng kết trung bình của sinh viên theo từng giảng viên phụ trách và tìm giảng viên có giá trị trung bình cao nhất. Chỉ xét các lượt đăng ký đã có đủ điểm.

Kỹ thuật sử dụng: CTE lọc lượt đủ điểm + GROUP BY giảng viên + AVG() + TOP 1 WITH TIES.

Câu lệnh SQL:

```
WITH DiemTheoGV AS (
    SELECT lhp.MaGVPT,
           (0.1*dk.DiemChuyenCan + 0.3*dk.DiemGiuaKy + 0.6*dk.DiemCuoiKy) AS
    DiemTongKet
    FROM DANG_KY dk
    JOIN LOP_HOC_PHAN lhp ON dk.MaLHP = lhp.MaLHP
    WHERE dk.DiemChuyenCan IS NOT NULL AND dk.DiemGiuaKy IS NOT NULL
    AND dk.DiemCuoiKy IS NOT NULL
)
SELECT TOP 1 WITH TIES gv.MaGV, gv.TenGV, AVG(d.DiemTongKet) AS
    DiemTongKetTrungBinh
FROM DiemTheoGV d
JOIN GIANG_VIEN gv ON gv.MaGV = d.MaGVPT
GROUP BY gv.MaGV, gv.TenGV
ORDER BY AVG(d.DiemTongKet) DESC;
```

Giải thích:

Việc lọc 'đã có đủ điểm' được thực hiện ngay từ bước CTE để AVG() không bị lệch do dữ liệu NULL từ các lượt đăng ký của học kỳ đang diễn ra.

Câu D28. GPA từng học kỳ; SV có GPA tăng nghiêm ngặt liên tiếp ≥ 3 học kỳ

Yêu cầu: Tính GPA từng học kỳ của từng sinh viên. Tìm sinh viên có GPA học kỳ tăng nghiêm ngặt qua ít nhất 3 học kỳ liên tiếp mà sinh viên có kết quả học tập.

Kỹ thuật sử dụng: CTE tính GPA theo học kỳ + LAG() so sánh với kỳ trước + kỹ thuật 'gom dãy liên tục' bằng SUM(cờ ngắt) OVER (... ROWS UNBOUNDED PRECEDING).

Câu lệnh SQL:

```
WITH GPAHocKy AS (
    SELECT dk.MaSV, lhp.NamHoc, lhp.HocKy,
           CAST(LEFT(lhp.NamHoc,4) AS INT)*10 + lhp.HocKy AS ThuTuThoiGian,

    SUM((0.1*dk.DiemChuyenCan+0.3*dk.DiemGiuaKy+0.6*dk.DiemCuoiKy)*hp.SoTinChi)
        / SUM(hp.SoTinChi) AS GPA_HocKy
    FROM DANG_KY dk
    JOIN LOP_HOC_PHAN lhp ON dk.MaLHP = lhp.MaLHP
    JOIN HOC_PHAN hp     ON lhp.MaHP = hp.MaHP
    WHERE dk.DiemChuyenCan IS NOT NULL AND dk.DiemGiuaKy IS NOT NULL
    AND dk.DiemCuoiKy IS NOT NULL
    GROUP BY dk.MaSV, lhp.NamHoc, lhp.HocKy
),
CoDauNgat AS (
    SELECT *,
           LAG(GPA_HocKy) OVER (PARTITION BY MaSV ORDER BY
ThuTuThoiGian) AS GPA_KyTruoc,
           ROW_NUMBER() OVER (PARTITION BY MaSV ORDER BY
ThuTuThoiGian) AS ThuTu
    FROM GPAHocKy
),
GanNhom AS (
    SELECT *,
           CASE WHEN GPA_KyTruoc IS NOT NULL AND GPA_HocKy >
GPA_KyTruoc THEN 0 ELSE 1 END AS CoNgat
    FROM CoDauNgat
),
NhomDay AS (
    SELECT *, SUM(CoNgat) OVER (PARTITION BY MaSV ORDER BY ThuTu
ROWS UNBOUNDED PRECEDING) AS NhomID
    FROM GanNhom
)
SELECT sv.MaSV, sv.TenSV, COUNT(*) AS SoHocKyTangLienTiep
FROM NhomDay n
JOIN SINH_VIEN sv ON sv.MaSV = n.MaSV
GROUP BY n.MaSV, sv.MaSV, sv.TenSV, n.NhomID
HAVING COUNT(*) >= 3;
```

Giải thích:

Đây là kỹ thuật kinh điển để tìm 'dãy con liên tiếp thỏa điều kiện' trong SQL: mỗi khi chuỗi tăng bị đứt quãng (GPA kỳ này không lớn hơn kỳ trước) thì gán cờ ngắt = 1; SUM() cộng dồn cờ ngắt theo thứ tự thời gian sẽ tạo ra 1 NhómID không đổi cho mỗi đoạn tăng liên tục; cuối cùng GROUP BY theo NhómID và đếm số dòng ≥ 3 sẽ tìm ra chuỗi tăng đủ dài. Khóa ThuTuThoiGian (năm*10+học kỳ) đảm bảo sắp đúng thứ tự thời gian thay vì sắp theo thứ tự chữ cái của NamHoc.

Câu D29. Tín chỉ tích lũy của từng SV; SV có tín chỉ tích lũy lớn nhất

Yêu cầu: Tính số tín chỉ tích lũy của từng sinh viên và tìm sinh viên có số tín chỉ tích lũy lớn nhất. Một học phần chỉ được tính tín chỉ 1 lần và chỉ khi sinh viên đã đạt học phần.

Kỹ thuật sử dụng: CTE BestAttempt (như D12) + lọc DiemTongKetCaoNhat ≥ 4 + SUM(SoTinChi) + TOP 1 WITH TIES.

Câu lệnh SQL:

```
-- (dùng lại CTE DiemDayDu, BestAttempt như câu D12)
TinChiTichLuy AS (
    SELECT MaSV, SUM(SoTinChi) AS TongTinChiTichLuy
    FROM BestAttempt
    WHERE DiemTongKetCaoNhat >= 4
    GROUP BY MaSV
)
SELECT TOP 1 WITH TIES sv.MaSV, sv.TenSV, t.TongTinChiTichLuy
FROM TinChiTichLuy t
JOIN SINH_VIEN sv ON sv.MaSV = t.MaSV
ORDER BY t.TongTinChiTichLuy DESC;
```

Giải thích:

Vì BestAttempt đã GROUP BY (MaSV, MaHP) nên mỗi học phần chỉ còn đúng 1 dòng (lần điểm cao nhất) cho mỗi sinh viên; lọc thêm điều kiện ĐẠT (≥ 4) trước khi SUM(SoTinChi) sẽ cho đúng số tín chỉ tích lũy, không đếm trùng học phần học lại.

Câu D30. SV đã đạt tất cả học phần bắt buộc trong CTĐT của ngành mình

Yêu cầu: Tìm sinh viên đã đạt tất cả các học phần bắt buộc trong chương trình đào tạo của ngành mà sinh viên đang theo học.

Kỹ thuật sử dụng: NOT EXISTS lồng nhau: 'không tồn tại học phần bắt buộc nào của ngành mà SV chưa đạt'.

Câu lệnh SQL:

```
SELECT sv.MaSV, sv.TenSV
FROM SINH_VIEN sv
JOIN LOP_HANH_CHINH lhc ON sv.MaLop = lhc.MaLop
WHERE NOT EXISTS (
```

```

SELECT 1 FROM CHUONG_TRINH_DAO_TAO ctdt
WHERE ctdt.MaNganh = lhc.MaNganh AND ctdt.LoaiHP = 'BB'
AND NOT EXISTS (
    SELECT 1 FROM DANG_KY dk
    JOIN LOP_HOC_PHAN lhp2 ON dk.MaLHP = lhp2.MaLHP
    WHERE dk.MaSV = sv.MaSV AND lhp2.MaHP = ctdt.MaHP
    AND dk.DiemChuyenCan IS NOT NULL AND dk.DiemGiuaKy IS NOT
NULL AND dk.DiemCuoiKy IS NOT NULL
    AND (0.1*dk.DiemChuyenCan+0.3*dk.DiemGiuaKy+0.6*dk.DiemCuoiKy)
    >= 4
)
);

```

Giải thích:

Cùng dạng 'lượng từ với mọi' như câu D17, nhưng áp dụng cho tập học phần bắt buộc (LoaiHP='BB') của đúng ngành sinh viên đang học, và điều kiện thỏa mãn ở tầng trong là 'đã đạt' (TK>=4) chứ không chỉ 'đã đăng ký'.

PHẦN E – BÀI TẬP NÂNG CAO / BONUS

Câu 31. Phát hiện các cặp lớp học phần có nguy cơ trùng phòng

Yêu cầu: Phát hiện các cặp lớp học phần có nguy cơ trùng phòng trong cùng học kỳ/năm học. Nếu mô hình được bổ sung thứ/tiết học, hãy phát hiện trùng lịch thực sự.

Kỹ thuật sử dụng: Self-join (tự kết) trên bảng LOP_HOC_PHAN với điều kiện a.MaLHP < b.MaLHP để mỗi cặp chỉ xuất hiện 1 lần.

Câu lệnh SQL:

```

SELECT a.MaLHP AS LopHocPhan1, b.MaLHP AS LopHocPhan2, a.PhongHoc,
a.HocKy, a.NamHoc
FROM LOP_HOC_PHAN a
JOIN LOP_HOC_PHAN b
    ON a.PhongHoc = b.PhongHoc
    AND a.HocKy = b.HocKy
    AND a.NamHoc = b.NamHoc
    AND a.MaLHP < b.MaLHP;

```

Giải thích:

Vì mô hình hiện tại CHƯA có cột thứ/tiết học, 2 lớp trùng phòng + cùng học kỳ/năm học chỉ là NGUY CƠ trùng lịch (điều kiện cần), chưa chắc trùng lịch thực sự (còn phụ thuộc thời khóa biểu cụ thể). Nếu bổ sung 2 cột Thu, Tiet vào LOP_HOC_PHAN, chỉ cần thêm 'AND a.Thu = b.Thu AND a.Tiet = b.Tiet' vào điều kiện JOIN để có kết quả chính xác tuyệt đối.

Câu 32. SV đăng ký học phần khi chưa đạt hết tiên quyết trực tiếp

Yêu cầu: Tìm những sinh viên đã đăng ký một học phần trong khi chưa đạt toàn bộ các học phần tiên quyết trực tiếp của học phần đó.

Kỹ thuật sử dụng: JOIN với HOC_PHAN_TIEN_QUYET + NOT EXISTS kiểm tra đã đạt tiên quyết chưa.

Câu lệnh SQL:

```
SELECT DISTINCT dk.MaSV, sv.TenSV, lhp.MaHP AS HocPhanDangKy
FROM DANG_KY dk
JOIN LOP_HOC_PHAN lhp ON dk.MaLHP = lhp.MaLHP
JOIN SINH_VIEN sv ON sv.MaSV = dk.MaSV
JOIN HOC_PHAN_TIEN_QUYET tq ON tq.MaHP = lhp.MaHP
WHERE NOT EXISTS (
    SELECT 1 FROM DANG_KY dk2
    JOIN LOP_HOC_PHAN lhp2 ON dk2.MaLHP = lhp2.MaLHP
    WHERE dk2.MaSV = dk.MaSV AND lhp2.MaHP = tq.MaHP_TienQuyet
    AND dk2.DiemChuyenCan IS NOT NULL AND dk2.DiemGiuKy IS NOT NULL
    AND dk2.DiemCuoiKy IS NOT NULL
    AND (0.1*dk2.DiemChuyenCan+0.3*dk2.DiemGiuKy+0.6*dk2.DiemCuoiKy)
    >= 4
);
```

Giải thích:

Với mỗi lượt đăng ký có học phần thuộc diện có tiên quyết, kiểm tra ứng với TỪNG tiên quyết trực tiếp (join với HOC_PHAN_TIEN_QUYET) xem sinh viên đó đã có lượt đạt ($TK \geq 4$) học phần tiên quyết đó chưa; nếu thiếu dù chỉ 1 tiên quyết, dòng đó sẽ được liệt kê.

Câu 33. Nâng cao câu 32: kiểm tra toàn bộ chuỗi tiên quyết trực tiếp + gián tiếp

Yêu cầu: Nâng câu trên: kiểm tra toàn bộ chuỗi học phần tiên quyết trực tiếp và gián tiếp bằng Recursive CTE.

Kỹ thuật sử dụng: Recursive CTE dựng 'bao đóng bắc cầu' (transitive closure) của quan hệ tiên quyết, rồi áp dụng lại logic NOT EXISTS của câu 32.

Câu lệnh SQL:

```
WITH TatCaTienQuyet AS (
    SELECT MaHP, MaHP_TienQuyet
    FROM HOC_PHAN_TIEN_QUYET
    UNION ALL
    SELECT t.MaHP, tq.MaHP_TienQuyet
    FROM TatCaTienQuyet t
    JOIN HOC_PHAN_TIEN_QUYET tq ON t.MaHP_TienQuyet = tq.MaHP
)
```

```

SELECT DISTINCT dk.MaSV, sv.TenSV, lhp.MaHP AS HocPhanDangKy
FROM DANG_KY dk
JOIN LOP_HOC_PHAN lhp ON dk.MaLHP = lhp.MaLHP
JOIN SINH_VIEN sv ON sv.MaSV = dk.MaSV
JOIN TatCaTienQuyet tq ON tq.MaHP = lhp.MaHP
WHERE NOT EXISTS (
    SELECT 1 FROM DANG_KY dk2
    JOIN LOP_HOC_PHAN lhp2 ON dk2.MaLHP = lhp2.MaLHP
    WHERE dk2.MaSV = dk.MaSV AND lhp2.MaHP = tq.MaHP_TienQuyet
    AND dk2.DiemChuyenCan IS NOT NULL AND dk2.DiemGiuaKy IS NOT NULL
    AND dk2.DiemCuoiKy IS NOT NULL
    AND (0.1*dk2.DiemChuyenCan+0.3*dk2.DiemGiuaKy+0.6*dk2.DiemCuoiKy)
    >= 4
)
OPTION (MAXRECURSION 100);

```

Giải thích:

CTE 'TatCaTienQuyet' đệ quy nối các cạnh tiên quyết liên tiếp để tìm ra MỌI học phần tiên quyết (trực tiếp và gián tiếp ở mọi mức) của 1 học phần, thay vì chỉ tiên quyết trực tiếp như câu 32. OPTION (MAXRECURSION 100) tăng giới hạn số vòng lặp đệ quy mặc định (100) — vẫn đủ cho dữ liệu bài toán vì chuỗi tiên quyết chỉ dài tối đa vài mức.

Câu 34. VIEW kết quả học tập tổng hợp của sinh viên

Yêu cầu: Viết một VIEW trả về: mã sinh viên, họ tên, tổng tín chỉ tích lũy, GPA tích lũy và số học phần bắt buộc còn thiếu trong chương trình đào tạo.

Kỹ thuật sử dụng: CREATE VIEW bọc quanh chuỗi CTE (BestAttempt) đã dùng ở các câu D12/D29, kết hợp thêm 1 CTE đếm học phần bắt buộc còn thiếu, rồi LEFT JOIN để không loại các SV chưa có điểm/chưa thiếu HP nào.

Câu lệnh SQL:

```

CREATE VIEW dbo.VW_KetQuaHocTap_SinhVien
AS
WITH DiemDayDu AS ( ... ),      -- như câu D12
BestAttempt AS ( ... ),        -- như câu D12
TongHop AS (
    SELECT MaSV,
        SUM(CASE WHEN DiemTongKetCaoNhat >= 4 THEN SoTinChi ELSE 0
        END) AS TongTinChiTichLuy,
        SUM(DiemTongKetCaoNhat*SoTinChi) / NULLIF(SUM(SoTinChi),0) AS
        GPA_TichLuy
    FROM BestAttempt GROUP BY MaSV
),

```

```
HocPhanBBConThieu AS (
    SELECT sv.MaSV, COUNT(*) AS SoHPBatBuocConThieu
    FROM SINH_VIEN sv
    JOIN LOP_HANH_CHINH lhc ON sv.MaLop = lhc.MaLop
    JOIN CHUONG_TRINH_DAO_TAO ct ON ct.MaNganh = lhc.MaNganh AND
    ct.LoaiHP = 'BB'
    WHERE NOT EXISTS (
        SELECT 1 FROM BestAttempt ba
        WHERE ba.MaSV = sv.MaSV AND ba.MaHP = ct.MaHP AND
        ba.DiemTongKetCaoNhat >= 4
    )
    GROUP BY sv.MaSV
)
SELECT sv.MaSV, sv.TenSV,
    ISNULL(th.TongTinChiTichLuy, 0) AS TongTinChiTichLuy,
    th.GPA_TichLuy,
    ISNULL(hp.SoHPBatBuocConThieu, 0) AS SoHocPhanBatBuocConThieu
FROM SINH_VIEN sv
LEFT JOIN TongHop th ON sv.MaSV = th.MaSV
LEFT JOIN HocPhanBBConThieu hp ON sv.MaSV = hp.MaSV;
```

Giải thích:

VIEW đóng gói lại toàn bộ logic tính GPA/tín chỉ tích lũy (đã trình bày ở câu D12, D29) và logic 'học phần bắt buộc còn thiếu' (biến thể của câu D30, đổi từ kiểm tra 'đã đạt hết chưa' sang ĐẾM số HP chưa đạt), giúp các truy vấn khác về sau chỉ cần SELECT trực tiếp từ VIEW thay vì lặp lại CTE phức tạp. Dùng LEFT JOIN + ISNULL để sinh viên chưa có điểm nào hoặc đã đạt hết vẫn xuất hiện với giá trị 0 thay vì bị loại khỏi kết quả.

Câu 35. Trigger/Stored procedure chặn đăng ký khi lớp đã đủ sĩ số

Yêu cầu: Viết trigger hoặc stored procedure bảo đảm khi thêm ĐĂNG KÝ, hệ thống từ chối nếu lớp đã đủ sĩ số. Thiết kế dữ liệu để kiểm thử cả trường hợp đăng ký thành công và bị từ chối.

Kỹ thuật sử dụng: Trigger INSTEAD OF INSERT trên bảng DANG_KY (đã cài đặt ở file RangBuoc_MU.sql, ràng buộc B.8); bổ sung thêm 2 trigger AFTER INSERT/UPDATE cho ràng buộc B.5 (trường bộ môn) và B.10 (chống chu trình tiên quyết, dùng bảng tạm chứa kết quả của Recursive CTE vì CTE không được lồng trực tiếp trong IF EXISTS()).

Câu lệnh SQL:

```
-- Trigger B.8 (sĩ số) — trích từ RangBuoc_MU.sql:
CREATE TRIGGER TRG_DANGKY_KIEMTRA_SISO
ON DANG_KY
```

```

INSTEAD OF INSERT
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (
        SELECT 1
        FROM (SELECT DISTINCT MaLHP FROM inserted) i
        JOIN LOP_HOC_PHAN I ON I.MaLHP = i.MaLHP
        WHERE (SELECT COUNT(*) FROM DANG_KY d WHERE d.MaLHP =
i.MaLHP)
            + (SELECT COUNT(*) FROM inserted i2 WHERE i2.MaLHP = i.MaLHP)
            > I.SiSoToiDa
    )
    BEGIN
        RAISERROR(N'Lớp học phần đã đủ số, không thể đăng ký thêm.', 16, 1);
        RETURN;
    END
    INSERT INTO DANG_KY (MaSV, MaLHP, NgayDangKy, DiemChuyenCan,
DiemGiuaKy, DiemCuoiKy)
    SELECT MaSV, MaLHP, NgayDangKy, DiemChuyenCan, DiemGiuaKy,
DiemCuoiKy FROM inserted;
END;

-- Trigger B.10 (chống chu trình tiên quyết) — dùng bảng tạm thay vì CTE trong IF
EXISTS:
CREATE TRIGGER dbo.TRG_HPTQ_KIEMTRA_CHUTRINH
ON HOC_PHAN_TIEN_QUYET
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    ;WITH DuongDi AS (
        SELECT MaHP, MaHP_TienQuyet FROM HOC_PHAN_TIEN_QUYET
        UNION ALL
        SELECT d.MaHP, t.MaHP_TienQuyet
        FROM DuongDi d JOIN HOC_PHAN_TIEN_QUYET t ON d.MaHP_TienQuyet
= t.MaHP
    )
    SELECT DISTINCT MaHP, MaHP_TienQuyet
    INTO #KetQuaChuTrinh
    FROM DuongDi WHERE MaHP = MaHP_TienQuyet
    OPTION (MAXRECURSION 100);

```

```

IF EXISTS (SELECT 1 FROM #KetQuaChuTrinh)
BEGIN
    DROP TABLE #KetQuaChuTrinh;
    RAISERROR(N'Phát hiện chu trình trong quan hệ tiên quyết học phần.', 16,
1);
    ROLLBACK TRANSACTION;
    RETURN;
END
DROP TABLE #KetQuaChuTrinh;
END;

```

Giải thích:

Trigger B.8 dùng `INSTEAD OF INSERT` (thay vì `AFTER INSERT`) vì cần **CHẶN** thao tác trước khi dữ liệu thật sự được ghi: cộng số đăng ký cũ trong bảng `DANG_KY` với số dòng đang chèn từ bảng ảo `'inserted'`, nếu vượt `SiSoToiDa` thì `RAISERROR` và không thực hiện `INSERT` thay thế bên dưới. Trigger B.10 gặp một giới hạn cú pháp quan trọng của T-SQL: CTE đệ quy **KHÔNG** được đặt trực tiếp bên trong `IF EXISTS(...)` — vì vậy phải tách thành 2 bước: (1) chạy Recursive CTE dựng bao đóng bắc cầu, đổ kết quả vào bảng tạm `#KetQuaChuTrinh`; (2) `IF EXISTS` kiểm tra trên bảng tạm đó. Nếu tồn tại cặp (`MaHP`, `MaHP_TienQuyét`) mà 2 giá trị bằng nhau, nghĩa là đi theo chuỗi tiên quyết từ một học phần có thể quay trở lại chính nó — tức có chu trình — và `ROLLBACK TRANSACTION` sẽ hủy bỏ thao tác `INSERT` vừa xảy ra. Dữ liệu kiểm thử ở mục 35.3 minh chứng cả 3 trigger đều có 1 trường hợp thành công và 1 trường hợp bị từ chối.